Homework 1

ASEN 5090 Introduction to GNSS

Justin Le
September 2, 2026

Problem 1

(a)

PL1 = 550 [m]
PL2 = 500 [m]

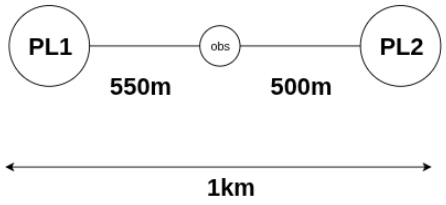

Figure 1: Ranging Diagram

Let

$$\rho^k = \sqrt{(x^k - x)^2 + (y^k - y)^2 + (z^k - z)^2} + b.$$

Given the setup, this can be constrained to a 1D problem on the x axis,

$$\rho^k = \sqrt{(x^k - x)^2} + b.$$

Which is equivalent to,

$$\rho^k = |(x^k - x)| + b.$$

Given the setup on a line, we have the ρ^1, the observed position be $550m$, and x^1 position on the line being 0. Likewise $\rho^2 = 500m$ and $x^2 = 1000m$. This gives a set of equations

PL1:

$$550m = |0 - x| + b$$
$$550m = x + b$$

PL2:

$$500m = |1000m - x| + b$$
$$-500m = -x + b$$

This can be solved by elimination, eliminating the bias b giving,

$$1050m = 2x$$

$$\boxed{\therefore \quad x = 525\,[m]}$$

Rewrite the PL1 equation to solve for b.

$$\boxed{\begin{aligned} \quad b &= 550m - x \\ \therefore \quad b &= 25\,[m] \end{aligned}}$$

b.

$$\begin{aligned} \text{PL1} &= 400\,[m] \\ \text{PL2} &= 1400\,[m] \end{aligned}$$

The solution method remains the same, where we develop a system of equations from the observed ranging measurements and based on the known locations of the sat.

PL1:

$$400m = |x| + b$$

PL2:

$$1400m = |1000 - x| + b$$

And with elimination and eliminating x this time, we obtain,

$$800m = 2b$$

$$\boxed{\therefore b = 400[m]}$$

And solve for x with

$$x = b - 400$$

$$\boxed{\therefore x = 0}$$

Problem 2

(a)

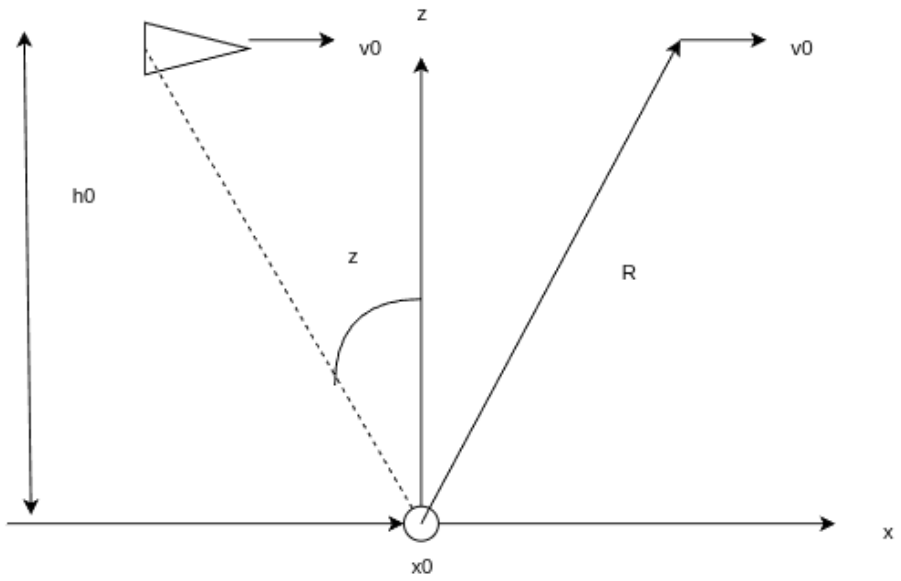

Figure 2: Ranging Diagram

From the diagram above, we are able to create solve for the following equations as functions of x_a, v_0, h_0.

$$R(x_a, v_0, h_0)$$
$$\dot{R}(x_a, v_0, h_0)$$
$$z(x_a, v_0, h_0)$$

When solving for range, one can simply use the pythagorean theorem given h_0 is constant.

$$\boxed{R = \sqrt{x_a^2 + h_0^2}}$$

Solving for range rate, take the differential w.r.t time.

$$\begin{aligned} R &= \sqrt{x_a^2 + h_0^2} \\ \dot{R} &= \frac{dR}{dt} \\ &= \frac{1}{2}\left(x_a^2 + h_0^2\right)^{-1/2}\left(2x_a\dot{x}_a + 2h_0\dot{h}_0\right) \\ &= \frac{x_a\dot{x}_a + h_0\dot{h}_0}{\sqrt{x_a^2 + h_0^2}} \end{aligned}$$

Since h_0 is constant, $\dot{h}_0 = 0$, and $\dot{x}_a = v_0$:

$$\boxed{\dot{R} = \frac{x_a v_0}{\sqrt{x_a^2 + h_0^2}}}$$

Solving for zenith, looking at the diagram, finding z as a function of x_a can simply be

$$\boxed{z = \arctan(\frac{x_a}{h_0})}$$

(b.)

See attached MATLAB script.

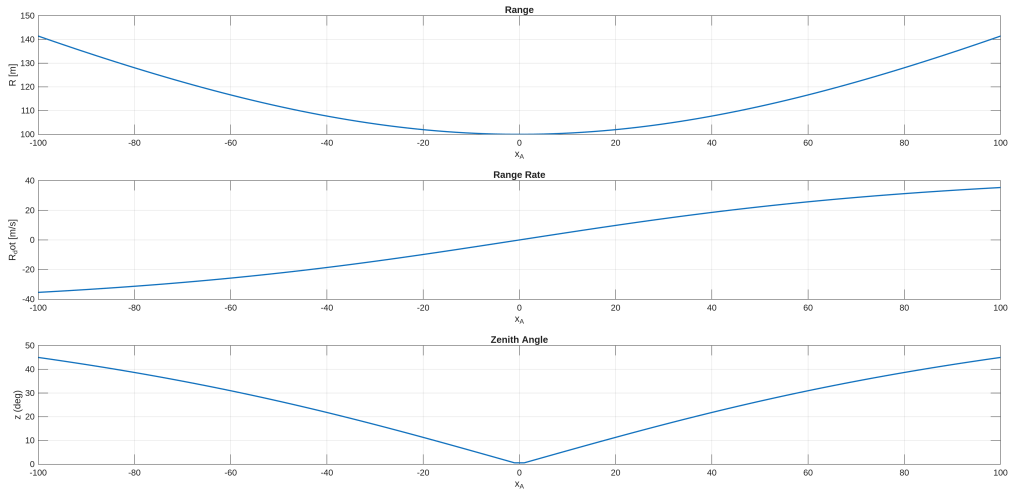

Figure 3: MATLAB Script Results

(c.)

The three types of measurements for determining x_a varies. Taking a look at the sensitivity of each measurement compared to the aircraft position. We see that there is different slopes at these different aircraft positions, when farther away from the ground station, Range rate and Zenith angle's slopes are more gradual, leading to less sensitivity. However when closer to the ground station, the pattern changes where Range rate and Zenith angle would have more steep slopes and be more sensitive. This is opposed to Range, where when close the ground station it would be less sensitive and more useful in this section.

(d.)

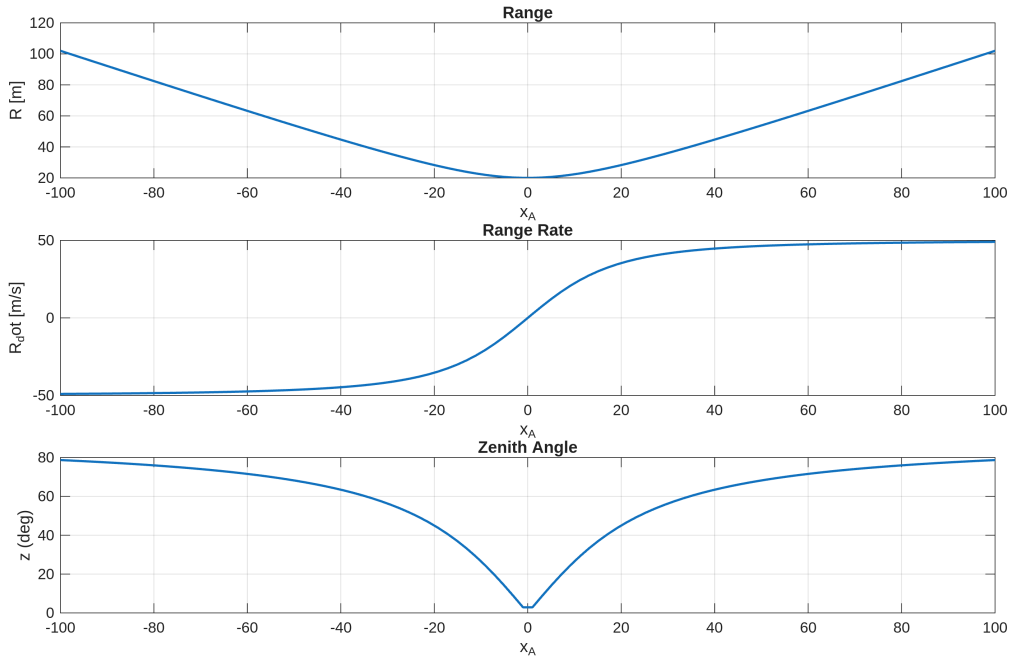

Figure 4: MATLAB Sensitivity at Lower Altitude

The sensitivity/utility of the observations at a lower altitude would depend on the parameter being examined and aircraft position. Taking a look at Figure 4, we can see the change in behavior, where when further away from the ground station, there is more sharper slopes for Range, but more smooth behavior from Range Rate and Zenith Angle. The is inverted as the aircraft gets closer to the ground station.

Problem 3

(a.)

```
firstHex_a =

    'E6D6'

lastHex_a =

    'E090'

XGiCA_code_diff =

     0

firstHex_c =

    'F8F4'

lastHex_c =

    '1368'

firstHex_d =

    '96C4'

lastHex_d =

    '6A72'
```

Figure 5: MATLAB C/A Code Generation Results

When examining the outputs for the first 16 chips expressed in hex, we do indeed obtain E6D6.

(b.)

When we create the C/A code form 1024 to 2046, we actually obtain the same code as we did in part (a.). This can be verified by taking the difference between the two vectors and then finding summing up all the elements, we obtain 0. This means they are identical.

(c.)

Repeating what we do for (a.), we are able to verify that it is the correct C/A code generated with the first 16 chips expressed in hex to be F8F4.

(d.)

Repeating what we do for (a.), we are able to verify that it is the correct C/A code generated with the first 16 chips expressed in hex to be 96C4.

Problem 4

(a.)

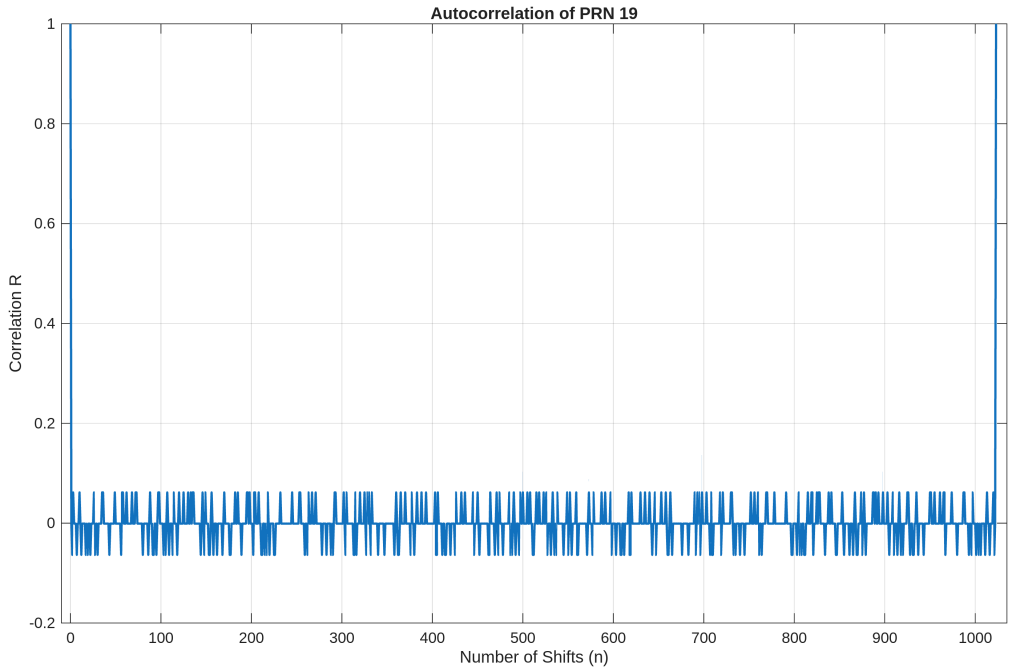

Figure 6: PRN 19 Auto Correlation

(b.)

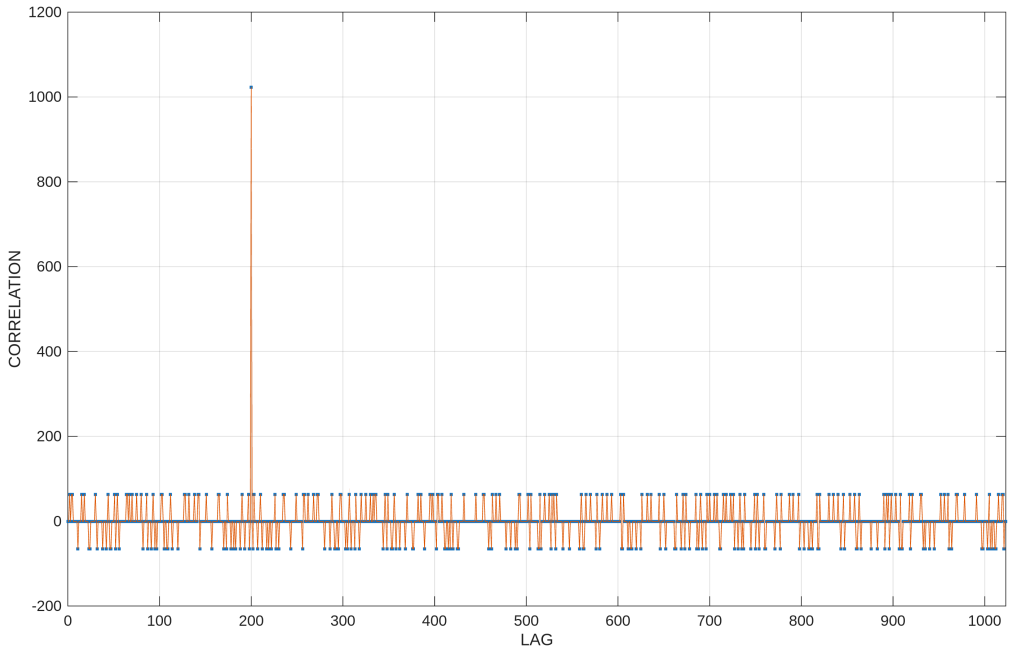

Figure 7: PRN 19 Delay Cross Correlation

Based on the Figure 7, results, we notice that there is a spike at 200 in the LAG axis. This is to be expected since this function takes the cross correlation between two codes, because this is the same code only shifted, the spike occurring at the amount we shifted it by is reasonable since at that point we would essentially be auto correlating the code resulting in similar behavior we observed in (a.).

(c.)

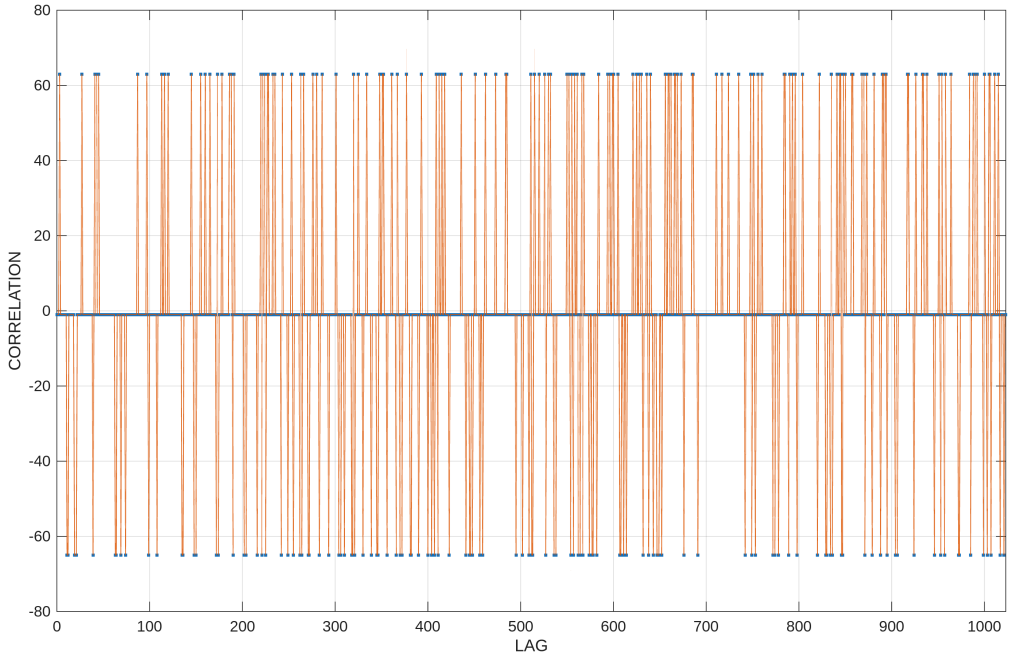

Figure 8: PRN 25 Cross Correlation

When comparing Figure 8 and the Figure 6 from part (a.), we observe a different behavior. We see instead of a stable variations of similar magnitude with a large spike where you would expect auto correlating behavior, there is instead only a stable variation of similar magnitude with no spike.

(d.)

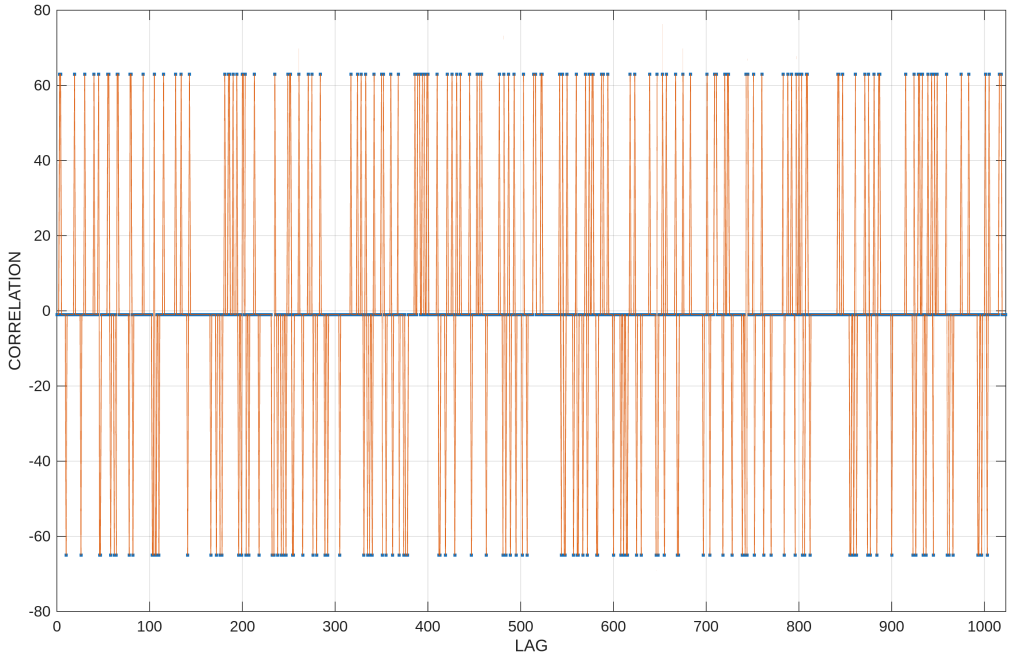

Figure 9: PRN 5 Cross Correlation

Similarly when comparing Figure 9 with Figure 6 from part (a)., we observe similar variations as we observed in part (c.). Where it is missing a spike and only has variations of similar magnitude.

(e.)

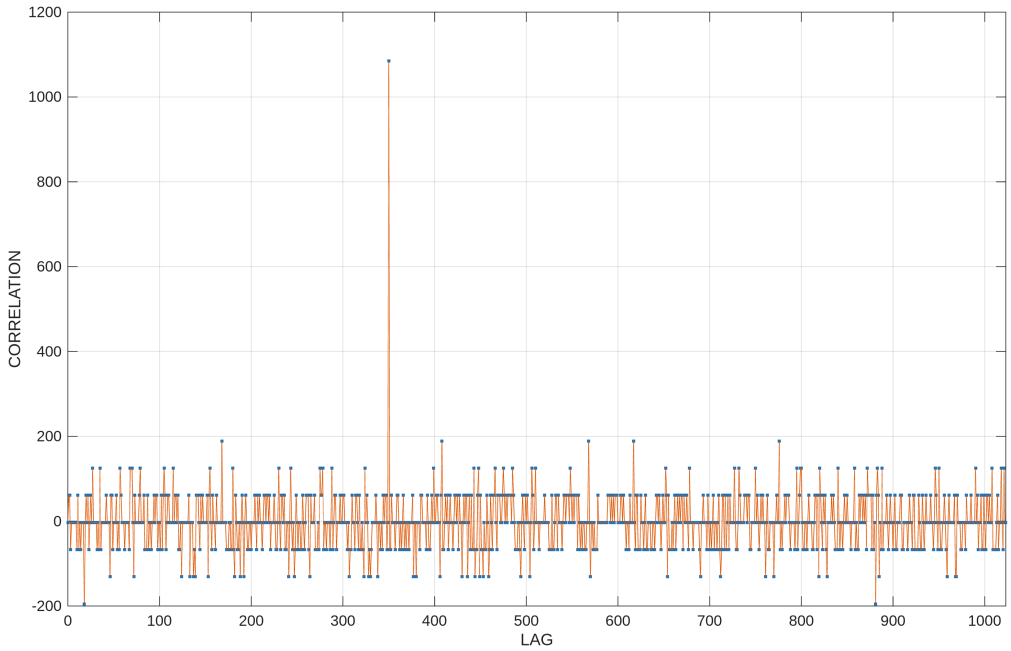

Figure 10: PRN Cross Correlation of sum of $x1, x2, x3$

The peak of the correlation within Figure 10 is indeed where I would expect it to be. Essentially we are mixing together multiple competing PRN signals from a GPS satellite. Because of how these codes are constructed, these codes have high auto correlation and low cross correlation with each other. Therefore, when these signals are mixed together and cross correlate with PRN 19, PRN 19 will have a spike where it would auto correlate to the respective chip shift on the lag axis, which in this case is 350.

(f.)

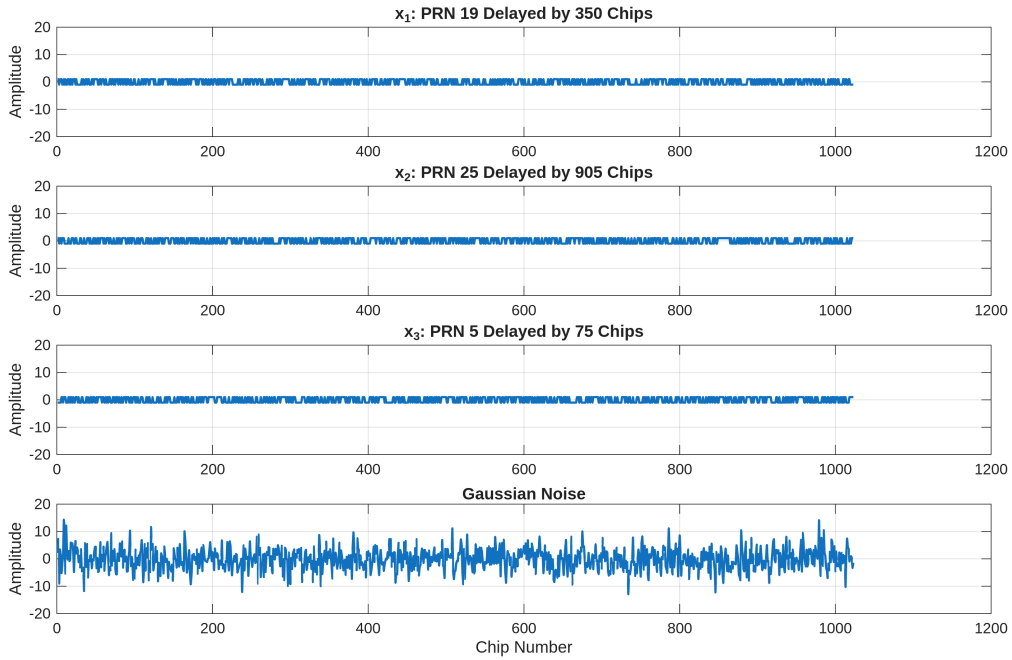

Figure 11: PRN Cross Correlation of sum of x_1, x_2, x_3

(g.)

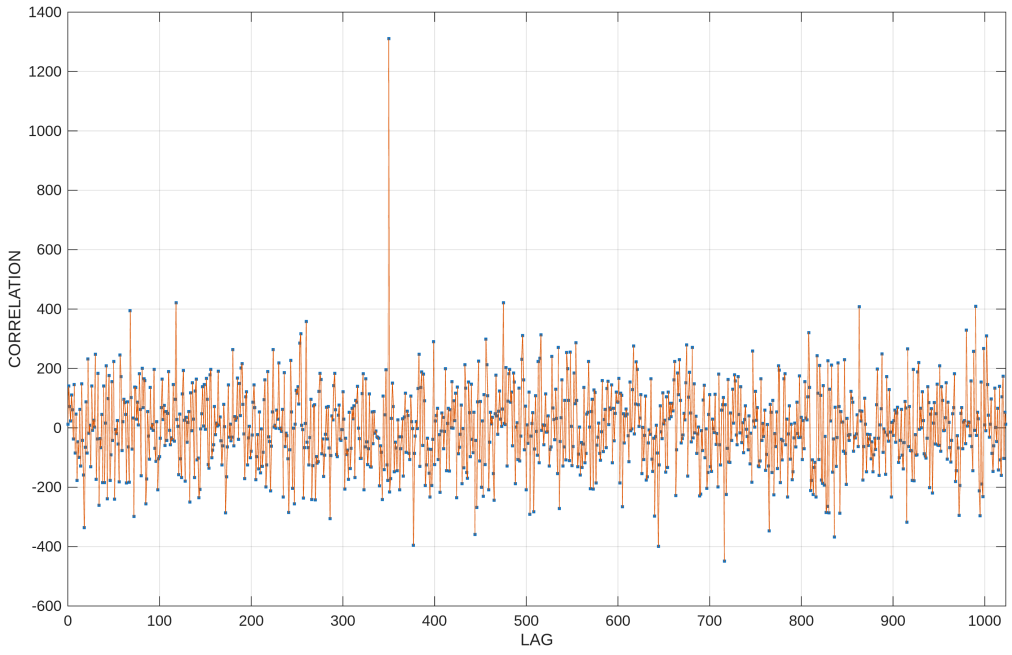

Figure 12: PRN Cross Correlation of sum of x_1, x_2, x_3 with Noise

Taking a look at the results at Figure 12, we do indeed observe that there is a noticeable peak at 350 in the lag axis. This is where I would expect it to be located, the same logic applies from part (f.), where there would naturally be a spike at this lag of 350 because that would be auto correlating with the original code of PRN 19 we are examining. I am however rather surprised how based on Figure 11, noise is comparatively very large compared to the chip values, and yet the correlated peak is still very prominent despite this large noise being applied.

MATLAB Code

```
%% ASEN 5090 GPS/GNSS
%% Justin Le
%% HW1 Problem 2 Script

clc; clear; close all

% Defining constants
v0 = 50; % [m/s]
x0 = 250; % [m]
h0 = 100; % [m]

% Equations are a function of xA
xa = linspace(-100,100);

% Ranging Equation
R = sqrt(xa.^2 + h0^2);

% Range Rate Equation
R_dot = xa.*v0.*(xa.^2+h0^2).^(-1/2);

% Zenith Equation
z = rad2deg(atan2(abs(xa),h0));


% Plotting Results
figure()

subplot(3,1,1)
plot(xa, R, 'LineWidth', 1.5)
grid on
xlabel('x_A')
ylabel('R [m]')
title('Range')

subplot(3,1,2)
plot(xa, R_dot, 'LineWidth', 1.5)
grid on
xlabel('x_A')
ylabel('R_dot [m/s]')
title('Range Rate')

subplot(3,1,3)
plot(xa, z, 'LineWidth', 1.5)
grid on
xlabel('x_A')
```

```
ylabel('z (deg)')
title('Zenith Angle')

% Save figure as PNG
exportgraphics(gcf, 'HW1_sensitivity.png', 'Resolution', 300);
```

```
%% ASEN 5090 GPS/GNSS
%% Justin Le
%% HW1 PRN GENERATOR

clc; clear; close all

% Load both G1 and G2 shift registers with all 1s
G1 = ones(1,10);
G2 = ones(1,10);

% Calcualate first bit output, then perform corresponding bit shifts
% for PRN19

XGiCA_code_a = prn_code_gen(3,6);

% Obtain first and last 16 bits
first16 = XGiCA_code_a(1:16);
last16 = XGiCA_code_a(end-15:end);

%Convert to hex for checking
firstHex_a = dec2hex(bin2dec(char(first16 + '0')), 4)
lastHex_a  = dec2hex(bin2dec(char(last16 + '0')), 4)

% This verifies that with the first 16 chips of PRN 19
% generated here is E6D6.

% b. Create C/A code output of PRN 19 from epoch 1024 to 2046

% simply take the existing generator and generate it to 2046.

XGiCA_code_b = zeros(1,1023);

for i = 1:2046
    % Phase Selector
    S1 = G2(3);
    S2 = G2(6);
    G2i = mod(S1+S2,2);

    %CA Code output
    XGiCA_code_b(i) = mod(G1(10)+G2i,2);

    % G1:
    G1_newBit = mod(G1(3)+G1(10),2);
    G1 = [G1_newBit G1(1:9)];

    % G2:
    G2_newBit = mod(G2(2)+G2(3)+G2(6)+G2(8)+G2(9)+G2(10),2);
```

```
    G2 = [G2_newBit G2(1:9)];
end

% Take the last part of the vector to obtain the XGiCA_code_b
XGiCA_code_b = XGiCA_code_b(end - 1022:end);
% Take difference and sum to check if both are identical
XGiCA_code_diff = sum(XGiCA_code_a-XGiCA_code_b)

% Find that XGiCA_code_diff = 0, therefore both are identical,
% this is expected as the codes should repeat after some time

% c. Repeat a for PRN 25.
% We can simply repeat what was done for a. and change the scheme

% Load both G1 and G2 shift registers with all 1s
G1 = ones(1,10);
G2 = ones(1,10);

% Calcualate first bit output, then perform corresponding bit shifts
% for PRN25

XGiCA_code_c = prn_code_gen(5,7);

% Obtain first and last 16 bits
first16 = XGiCA_code_c(1:16);
last16 = XGiCA_code_c(end-15:end);

%Convert to hex for checking
firstHex_c = dec2hex(bin2dec(char(first16 + '0')), 4)
lastHex_c  = dec2hex(bin2dec(char(last16 + '0')), 4)

% d. Repeat a for PRN 5.

% Load both G1 and G2 shift registers with all 1s
G1 = ones(1,10);
G2 = ones(1,10);

% Calcualate first bit output, then perform corresponding bit shifts
% for PRN25

XGiCA_code_d = prn_code_gen(1,9);

% Obtain first and last 16 bits
first16 = XGiCA_code_d(1:16);
last16 = XGiCA_code_d(end-15:end);

%Convert to hex for checking
firstHex_d = dec2hex(bin2dec(char(first16 + '0')), 4)
```

```
lastHex_d  = dec2hex(bin2dec(char(last16 + '0')), 4)

%% Part 4. Correlation
% a. plot the auro correlation function of PRN 19

% Calculate the auto correlation

n_shift = 0:1:1023;
R_n_auto = zeros(1,1024);

% Convert into -1 and 1s
XGiCA_code = (XGiCA_code_a==0)*(1) + (XGiCA_code_a==1)*(-1);

% On first iteration there should be no shift
XGiCA_code_shift = XGiCA_code;
for n = 1:1:length(n_shift)
    % Cache code to be shifted during loop
    XGiCA_code_cache = XGiCA_code_shift;
    % Auto correlation function
    R_n_auto(n) = (1/1023) * sum(XGiCA_code.*(XGiCA_code_cache));
    % Shifting by 1
    XGiCA_code_shift = XGiCA_code_shift([end (1:end-1)]);
end

figure()
plot(n_shift, R_n_auto, LineWidth=1.5)
xlabel('Number of Shifts (n)')
ylabel('Correlation R')
title('Autocorrelation of PRN 19')
xlim([-10, 1035])
grid on

exportgraphics(gcf, 'PRN19_Autocorrelation.png', 'Resolution', 300);

% b.
% delay by 200 chips
XGiCA_code_delay_200 = XGiCA_code([end-199:end 1:end-200]);
XGiCA_code_PRN19 = XGiCA_code;
figure()
cyc_corr_basic(XGiCA_code_delay_200, XGiCA_code_PRN19)
exportgraphics(gcf, 'PRN19_PRN19_delay_200_Crosscorrelation.png', 'Resolution', 300);

% c.
% PRN 25 vs PRN 19

% Convert code to 1s and -1s for PRN 25
XGiCA_code_PRN25 = (XGiCA_code_c==0)*(1) + (XGiCA_code_c==1)*(-1);
figure()
```

```
cyc_corr_basic(XGiCA_code_PRN25, XGiCA_code_PRN19)
exportgraphics(gcf, 'PRN25_PRN19_Crosscorrelation.png', 'Resolution', 300);

% d.
% PRN 5 vs PRN 19

% Convert code to 1s and -1s for PRN 5
XGiCA_code_PRN5 = (XGiCA_code_d==0)*(1) + (XGiCA_code_d==1)*(-1);
figure()
cyc_corr_basic(XGiCA_code_PRN5, XGiCA_code_PRN19)
exportgraphics(gcf, 'PRN5_PRN19_Crosscorrelation.png', 'Resolution', 300);

% e.
% Create three 1023 chip PRNs
% x1 = PRN19 delayed by 350
% x2 = PRN25 delayed by 905
% x3 = PRN5  delayed by 75

x1 = XGiCA_code_PRN19([end-349:end 1:end-350]);
x2 = XGiCA_code_PRN25([end-904:end 1:end-905]);
x3 = XGiCA_code_PRN5([end-74:end 1:end-75]);

% Sum these PRNs
x_sum = x1+x2+x3;

% Correlate with PRN19
figure()
cyc_corr_basic(x_sum, XGiCA_code_PRN19)
exportgraphics(gcf, 'PRN5xsum_PRN19_Crosscorrelation.png', 'Resolution', 300);

% f.
% Creating noise
noise = 4*randn(1,1023);

% Plotting with subplots x1,x2,x3, and noise
figure()

subplot(4,1,1)
plot(1:length(x1), x1, LineWidth=1.5)
title('x_1: PRN 19 Delayed by 350 Chips')
ylabel('Amplitude')
ylim([-20 20])
grid on

subplot(4,1,2)
plot(1:length(x2), x2, LineWidth=1.5)
title('x_2: PRN 25 Delayed by 905 Chips')
ylabel('Amplitude')
```

```
ylim([-20 20])
grid on

subplot(4,1,3)
plot(1:length(x3), x3, LineWidth=1.5)
title('x_3: PRN 5 Delayed by 75 Chips')
ylabel('Amplitude')
ylim([-20 20])
grid on

subplot(4,1,4)
plot(1:length(noise), noise, LineWidth=1.5)
title('Gaussian Noise')
xlabel('Chip Number')
ylabel('Amplitude')
ylim([-20 20])
grid on
exportgraphics(gcf, 'subplot_x1x2x3_noise.png', 'Resolution', 300);

% g.
% Sum x1,x2,x3, and noise
x_sum_noise = x1+x2+x3+noise;
figure()
cyc_corr_basic(x_sum_noise, XGiCA_code_PRN19)
exportgraphics(gcf, 'PRN5xsum_noise_PRN19_Crosscorrelation.png', 'Resolution', 300);

function XGiCA_code = prn_code_gen(S1_chip, S2_chip)

% Load both G1 and G2 shift registers with all 1s
G1 = ones(1,10);
G2 = ones(1,10);

% Calcualate first bit output, then perform corresponding bit shifts
% for PRN19

XGiCA_code = zeros(1,1023);

for i = 1:1023
    % Phase Selector as a result from function inputs
    S1 = G2(S1_chip);
    S2 = G2(S2_chip);
    G2i = mod(S1+S2,2);

    %CA Code output
    XGiCA_code(i) = mod(G1(10)+G2i,2);

    % G1:
    G1_newBit = mod(G1(3)+G1(10),2);
```

```
    G1 = [G1_newBit G1(1:9)];

    % G2:
    G2_newBit = mod(G2(2)+G2(3)+G2(6)+G2(8)+G2(9)+G2(10),2);
    G2 = [G2_newBit G2(1:9)];
end

end

function P = cyc_corr_basic(x,y)
% Function to compute the cyclic correlation for ASEN 5090
% First input is the "received" signal
% Second input is the "replica" signal - this is the one that shifts
n = length(x);
x = reshape(x,1,n);
yshifted = reshape(y,n,1);
% make sure x is a row vector
% make the replica of y a column vector
lag = [0:n]; Rxy = NaN(size(lag));
for i = 1:length(lag) % sweep the lag from 0 to n
Rxy(i) = x*yshifted;
yshifted = [yshifted(end); yshifted(1:end-1)];
end
plot(lag,Rxy,'.',lag,Rxy,'-'),grid,xlim([0 n])
xlabel('LAG'),ylabel('CORRELATION')
```